

THE CASE STATEMENT

CASE

Allows you to process a series of IF/THEN logical operators in a specific order

CASE WHEN logic1 **THEN** value1 **WHEN** logic2 **THEN** value2 **ELSE** value3 **END**

Every CASE statement begins with **CASE**, ends with **END**, and contains at least one **THEN/WHEN** pair.

This is where you define your logical operators and the values you want assigned when met.

Example:

```
CASE  
  WHEN category IN ('horror', 'suspense') THEN 'too scary'  
  WHEN length > 90 THEN 'too long'  
  ELSE 'we should see it'  
END
```

HEY THIS IS IMPORTANT!

CASE statements execute in the order they appear; if a record satisfies more than one logical condition, the record will be assigned by the **first THEN statement**

CASE OPERATORS

Operator	What it Means
=	Equals
<>	Does NOT Equal
>	Greater Than
<	Less Than
>=	Greater Than Or Equal To
<=	Less Than Or Equal To
BETWEEN	A Range Between Two Values
LIKE	Matching a Pattern Like This
IN()	Equals One of These Values

PRO TIP: “PIVOTING” DATA WITH COUNT & CASE

CASE “PIVOTS”

Excel’s ability to pivot to columns can be replicated in SQL using **COUNT** and **CASE**

Excel makes it very easy to “pivot” data on two dimensions.

Here we’re breaking down the count of **inventory_id** by **film_id** (rows) and **store_id** (columns) to quickly see how many copies of each film we have at each store:

	inventory...	film_id	store_id	last_update
1	1	1	1	2006-02-15 05:09:17
2	1	1	1	2006-02-15 05:09:17
3	1	1	1	2006-02-15 05:09:17
4	1	1	1	2006-02-15 05:09:17
5	1	1	2	2006-02-15 05:09:17
6	1	1	2	2006-02-15 05:09:17
7	1	1	2	2006-02-15 05:09:17
8	1	1	2	2006-02-15 05:09:17
9	2	2	2	2006-02-15 05:09:17
10	2	2	2	2006-02-15 05:09:17
11	2	2	2	2006-02-15 05:09:17
12	3	2	2	2006-02-15 05:09:17
13	3	2	2	2006-02-15 05:09:17
14	3	2	2	2006-02-15 05:09:17
15	3	2	2	2006-02-15 05:09:17
16	4	1	1	2006-02-15 05:09:17
17	4	1	1	2006-02-15 05:09:17
18	4	1	1	2006-02-15 05:09:17
19	4	1	1	2006-02-15 05:09:17

Row Labels	1	2	Grand Total
1	4	4	8
2	3	3	6
3	4	4	8
4	4	3	7
5	3	3	6
6	3	3	6
7	2	3	5
8	4	4	8
9	3	2	5
10	4	3	7
11	4	3	7
12	3	4	7
13	3	4	7
14	2	4	6
15	2	2	4
16	2	2	4
17	3	3	6
18	3	3	6
19	4	2	6
20	3	3	6
21	2	4	6
22	4	3	7
23	3	2	5
24	4	4	8
25	4	2	6
26	2	3	5
27	4	4	8
28	3	3	6
29	2	2	4
30	2	2	4
31	4	4	8
32	2	2	4
33	4	4	8
34	4	3	7
35	4	3	7
36	4	3	7
37	4	3	7

Both methods yield identical results

MySQL can do the same thing using **COUNT** functions inside of a **CASE** statement:

film_id	store_1_copies	store_2_copies	total_copies
1	4	4	8
2	0	3	3
3	0	4	4
4	4	3	7
5	0	3	3
6	3	3	6
7	2	3	5
8	0	4	4
9	3	2	5
10	4	3	7
11	4	3	7
12	3	4	7
13	0	4	4
14	2	4	6
15	2	2	4
16	2	2	4
17	3	3	6
18	3	3	6
19	4	2	6
20	3	0	3
21	2	4	6
22	4	3	7

CASE & COUNT

- Excel makes it easy to pivot data into columns and rows
- We can do the same thing in MySQL by using GROUP BY, combined with the “CASE Pivot”
- When CASE Pivoting, we use COUNT() and only count records that match a certain criteria

PRO TIP:

Use **GROUP BY** to define your row labels, and **CASE** to pivot to columns

MySQL QUERY IN ACTION:

```
SELECT
  film_id,
  COUNT(CASE WHEN store_id = 1 THEN inventory_id ELSE NULL END) AS store_1_copies,
  COUNT(CASE WHEN store_id = 2 THEN inventory_id ELSE NULL END) AS store_2_copies,
  COUNT(inventory_id) AS total_copies
FROM inventory
GROUP BY
  film_id
ORDER BY
  film_id;
```

ORIGINAL TABLE:

inventory...	film_id	store_id	last_update
1	1	1	2006-02-15 05:09:17
2	1	1	2006-02-15 05:09:17
3	1	1	2006-02-15 05:09:17
4	1	1	2006-02-15 05:09:17
5	1	2	2006-02-15 05:09:17
6	1	2	2006-02-15 05:09:17
7	1	2	2006-02-15 05:09:17
8	1	2	2006-02-15 05:09:17
9	2	2	2006-02-15 05:09:17
10	2	2	2006-02-15 05:09:17
11	2	2	2006-02-15 05:09:17
12	3	2	2006-02-15 05:09:17
13	3	2	2006-02-15 05:09:17
14	3	2	2006-02-15 05:09:17
15	3	2	2006-02-15 05:09:17
16	4	1	2006-02-15 05:09:17
17	4	1	2006-02-15 05:09:17
18	4	1	2006-02-15 05:09:17
19	4	1	2006-02-15 05:09:17

QUERY RESULTS:

film_id	store_1_copies	store_2_copies	total_copies
1	4	4	8
2	0	3	3
3	0	4	4
4	4	3	7
5	0	3	3
6	3	3	6
7	2	3	5
8	0	4	4
9	3	2	5
10	4	3	7
11	4	3	7
12	3	4	7
13	0	4	4
15	2	4	6
16	2	2	4
17	3	3	6
18	3	3	6
19	4	2	6
20	3	0	3
21	2	4	6
22	4	3	7
23	3	2	5

COMMON JOIN TYPES

INNER JOIN

Returns records that exist in **BOTH** tables, and excludes unmatched records from either table

FROM leftTableName
INNER JOIN rightTableName

LEFT JOIN

Returns ALL records from the **LEFT** table, and any matching records from the **RIGHT** table

FROM leftTableName
LEFT JOIN rightTableName

RIGHT JOIN

Returns ALL records from the **RIGHT** table, and any matching records from the **LEFT** table

FROM leftTableName
RIGHT JOIN rightTableName

FULL OUTER JOIN

Returns ALL records from **BOTH** tables, including non-matching records

FROM leftTableName
FULL JOIN rightTableName

UNION

Returns all data from one table, with all data from another table **appended to the end**

SELECT FROM firstName
UNION
SELECT FROM secondTableName

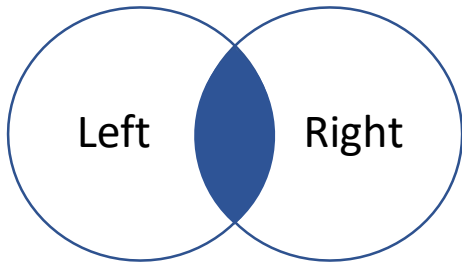
INNER JOIN

INNER JOIN

Returns records from BOTH tables when there is a match, and excludes unmatched records

INNER JOIN rightTableName **ON** leftTable.columnName = rightTable.columnName

Tells SQL to return only the **overlap**



This is where you name your right table, and tell SQL which column to match on by specifying the column name from each table

Example:

```
FROM rental
INNER JOIN customer
ON rental.customer_id = customer.customer_id
```

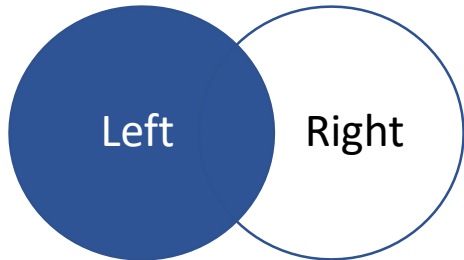
LEFT JOIN

LEFT JOIN

Returns all records from the LEFT table, and any matched records from the RIGHT table

LEFT JOIN rightTableName **ON** leftTable.columnName = rightTable.columnName

*Tells SQL to return the overlap and everything from the **left** table*



This is where you name your right table, and tell SQL which column to match on by specifying the column name from each table

Example:

```
FROM rental  
LEFT JOIN customer  
ON rental.customer_id = customer.customer_id
```

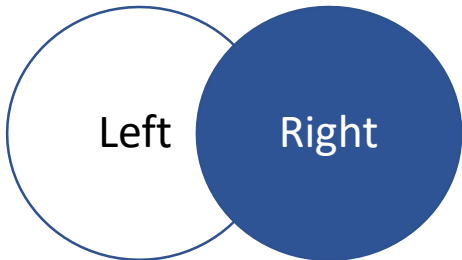
RIGHT JOIN

RIGHT JOIN

Returns all records from the RIGHT table, and any matched records from the LEFT table

RIGHT JOIN rightTableName **ON** leftTable.columnName = rightTable.columnName

*Tells SQL to return the overlap and everything from the **right** table*



This is where you tell SQL how to look for a match by specifying columns from each table

Example:

```
FROM rental  
RIGHT JOIN customer  
ON rental.customer_id = customer.customer_id
```

FULL OUTER JOIN (aka FULL JOIN)

FULL JOIN

Returns all records from BOTH tables when there is a match in either one of the tables

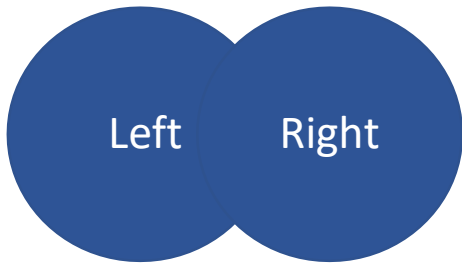
FULL JOIN rightTableName ON leftTable.columnName = rightTable.columnName

Tells SQL to return records from both tables

This is where you tell SQL how to look for a match, by specifying columns from each table

Example:

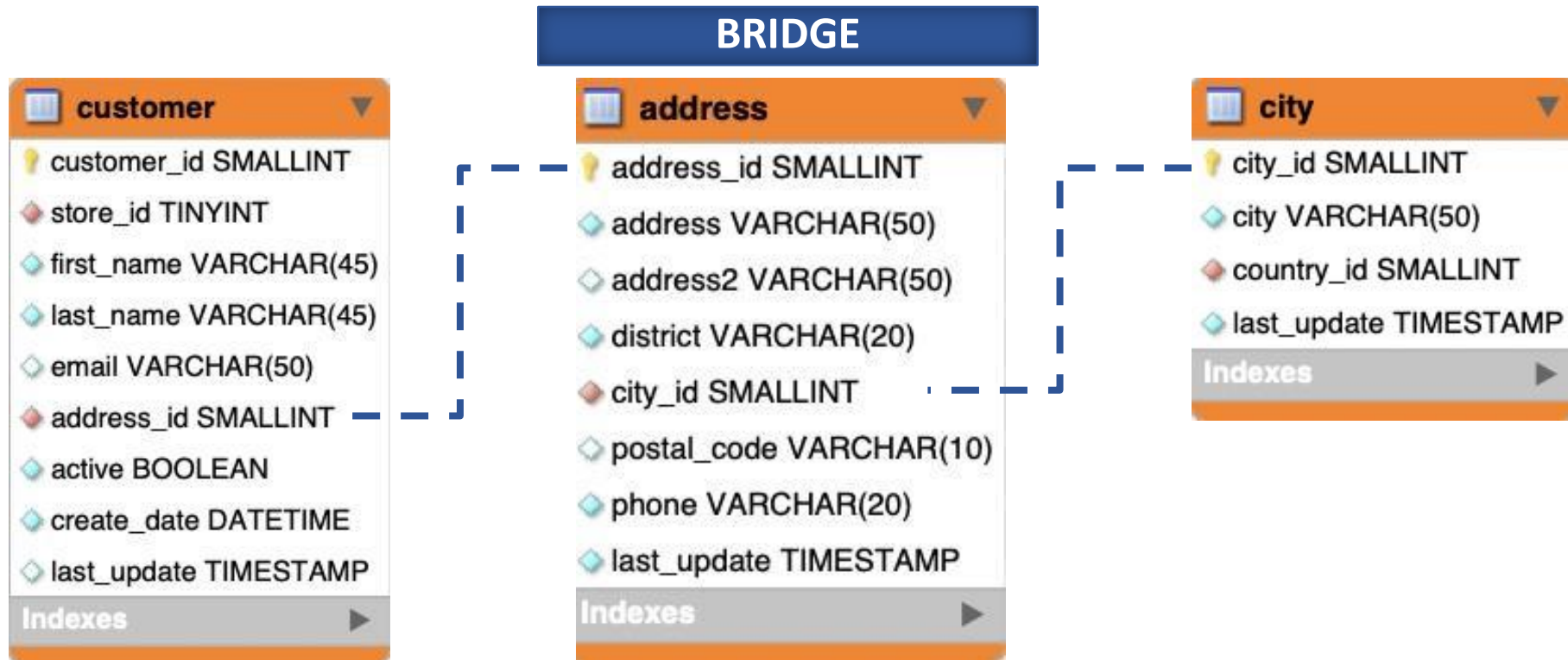
```
FROM rental
FULL JOIN customer
ON rental.customer_id = customer.customer_id
```



PRO TIP: “BRIDGING” UNRELATED TABLES

When you need to connect two tables which **do not directly relate**, look for a third table containing keys common to both; this can serve as a “**bridge**” to join your tables together

- Here we have no key to connect the **customer** table directly to **city**, but we can join **customer** to **address** (using *address_id*), and **address** to **city** (using *city_id*). In this case the **address** table serves as our bridge.



PRO TIP: BRIDGING

- In this example, we need the **film.title** and **category.name** fields, but the **film** and **category** tables don't have a matching key
- We'll need to find a "**bridge**" table, which relates to both of the tables we want to pull data from
- In this example, **film_category** has keys to both **film** and **category** tables, and can serve as our bridge

PRO TIP:



If you aren't sure if two tables can connect, list the tables each one is joined to. Find a bridge!

MySQL QUERY IN ACTION:

```
SELECT
    film.film_id,
    film.title,
    film_category.category_id,
    category.name

FROM film
    INNER JOIN film_category
        ON film.film_id = film_category.film_id
    INNER JOIN category
        ON film_category.category_id = category.category_id

ORDER BY film_id
```

QUERY RESULTS:

Result Grid			 Filter Rows:	Q Search
film_id	title	category_id	name	
▶ 1	ACADEMY DINOSAUR	6	Documentary	
2	ACE GOLDFINGER	11	Horror	
3	ADAPTATION HOLES	6	Documentary	
4	AFFAIR PREJUDICE	11	Horror	
5	AFRICAN EGG	8	Family	
6	AGENT TRUMAN	9	Foreign	
7	AIRPLANE SIERRA	5	Comedy	
8	AIRPORT POLLOCK	11	Horror	
9	ALABAMA DEVIL	11	Horror	
10	ALADDIN CALENDAR	15	Sports	
11	ALAMO VIDEOTAPE	9	Foreign	
12	ALASKA PHANTOM	12	Music	
13	ALI FOREVER	11	Horror	
14	ALICE FANTASIA	4	Classics	
15	ALIEN CENTER	9	Foreign	

Using two JOINS, we can produce a result set containing values from 3 tables:

- **film**
- **film_category**
- **category**

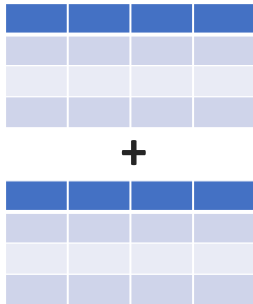
UNION: A DIFFERENT WAY TO COMBINE TABLES

UNION

Returns all data from the FIRST table, with all data from the SECOND table appended to the end

UNION SELECT sameColumnName **FROM** secondTableName

*UNION tells SQL to combine the results of one **SELECT** statement with another*



*Instead of a JOIN to another table, we write a second **SELECT** statement here:*

Example:

```
-- yields one column with all names (first+last)
SELECT first_name, last_name FROM advisor
UNION
SELECT first_name, last_name FROM investor
```